

One Pass, Any Order: Position-Invariant Listwise Reranking for LLM-Based Recommendation

Ethan Bito
RMIT University
Melbourne, Australia
s4102812@student.rmit.edu.au

Yongli Ren
RMIT University
Melbourne, Australia
yongli.ren@rmit.edu.au

Estrid He
RMIT University
Melbourne, Australia
estr.id.he@rmit.edu.au

Abstract

Large language models (LLMs) are increasingly used for recommendation reranking, but their listwise predictions can depend on the order in which candidates are presented. This creates a mismatch between the set-based nature of recommendation and the sequence-based computation of decoder-only LLMs, where permuting an otherwise identical candidate set can change item scores and final rankings. Such order sensitivity makes LLM-based rerankers difficult to rely on, since rankings may reflect prompt serialization rather than user preference. We propose InvariRank, a permutation-invariant listwise reranking framework that addresses this dependence at the architectural level. InvariRank blocks cross-candidate attention with a structured attention mask, and negates position-induced scoring changes through shared positional framing under Rotary Positional Embeddings (RoPE). Combined with a listwise learning-to-rank objective, the model scores all candidates in a single forward pass, avoiding permutation-based invariance training objectives which require multiple permutations of a candidate set. Experiments on recommendation benchmarks show that InvariRank maintains competitive ranking effectiveness while producing stable rankings across candidate permutations. The results suggest that architectural invariance is a practical route to reliable and efficient LLM-based recommendation reranking. The source code is at <https://github.com/ejbrito/InvariRank>.

CCS Concepts

• Information systems → Recommender systems; Retrieval models and ranking.

Keywords

recommender systems, large language models, listwise ranking, permutation invariance, position bias

ACM Reference Format:

Ethan Bito, Yongli Ren, and Estrid He. 2026. One Pass, Any Order: Position-Invariant Listwise Reranking for LLM-Based Recommendation. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '26)*, July 20–24, 2026, Melbourne, VIC, Australia. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3805712.3809952>



This work is licensed under a Creative Commons Attribution 4.0 International License. *SIGIR '26, Melbourne, VIC, Australia*
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2599-9/2026/07
<https://doi.org/10.1145/3805712.3809952>

1 Introduction

Large language models (LLMs) are increasingly used for listwise reranking in recommender systems, as they combine item descriptions, user histories, and task instructions within a single conditional scoring process [7, 10, 12, 25]. In two-stage pipelines, a fixed set of retrieved candidates is passed to the LLM and reranked in one forward pass, making listwise LLM reranking a natural and flexible design choice.

However, this use of LLMs introduces a fundamental mismatch. Recommendation candidates form a set, but decoder-only LLMs process them as a sequence. As a result, rankings can depend not only on user preference and item relevance, but also on the arbitrary order in which candidates are serialized. Even when the user history and candidate set are unchanged, simply permuting the candidate order can produce different scores and rankings [1, 23]. This goes beyond prompt sensitivity, such that the ranker may fail to behave as a well-defined ranking function over a candidate set.

Such order dependence creates a serious reliability problem for LLM-based recommendation. A model that promotes an item in one serialization but demotes it in another is unstable and difficult to evaluate, reproduce, and deploy. Standard ranking metrics can obscure this issue because they measure the quality of a single observed ranking, while ignoring whether the same candidate set would produce a different ranking under an equivalent presentation. For listwise LLM reranking to be reliable, ranking quality and permutation robustness must both be considered. To address this, we enforce permutation invariance directly at the architectural level.

We argue that this sensitivity arises from two architectural sources in decoder-only LLMs. First, cross-candidate attention leakage allows tokens from one candidate to attend to tokens from others, creating order-dependent interference between items. Second, offset drift under Rotary Positional Embeddings (RoPE) changes the relative positional offsets between candidates and the shared user context when candidates are permuted, perturbing attention patterns and item scores [21, 23]. Together, these mechanisms couple predicted relevance to candidate position rather than only to candidate content and user context.

Prior work can reduce this behavior, but generally leaves the source of order dependence intact. Inference-time methods such as permutation ensembling improve stability by aggregating multiple candidate orders, but require repeated model calls [13, 23]. Comparison-based prompting avoids some listwise ordering effects by decomposing ranking into smaller decisions, but increases inference cost [1, 2]. Training-time regularization penalizes disagreement across permutations, but the model can still access order information during computation [2].

We instead remove order-dependent channels architecturally. We design the computation so that each candidate is scored from the shared user context and its own content, while preventing other candidates from influencing its representation. We further use a shared positional framing under RoPE so that each candidate is evaluated under the same positional relationship to the user context, regardless of its position in the serialized input [5, 21]. This makes permutation invariance a property of the model computation rather than an effect of repeated inference or auxiliary loss terms.

We make the following contributions. (1) We identify cross-candidate attention leakage and RoPE-induced offset drift as two concrete sources of position bias in decoder-only LLM reranking. (2) We propose InvariRank, a permutation-invariant, single-pass listwise reranker that enforces set-level invariance by design, while retaining listwise learning through a LambdaRank objective. (3) We show that InvariRank maintains competitive ranking effectiveness while substantially improving permutation robustness across benchmarks, providing a practical path toward reliable LLM-based recommendation reranking.

2 Related Work

In recommender systems [9, 27, 30], LLMs are commonly deployed as rerankers in two-stage pipelines, where a retrieval model first produces a fixed candidate set because of computational and context-length constraints [6, 17, 22, 26]. Among pointwise, pairwise, and listwise formulations, listwise reranking has emerged as a particularly effective paradigm for LLMs, as it enables joint scoring of all candidates and leverages global context [19, 20, 28, 31].

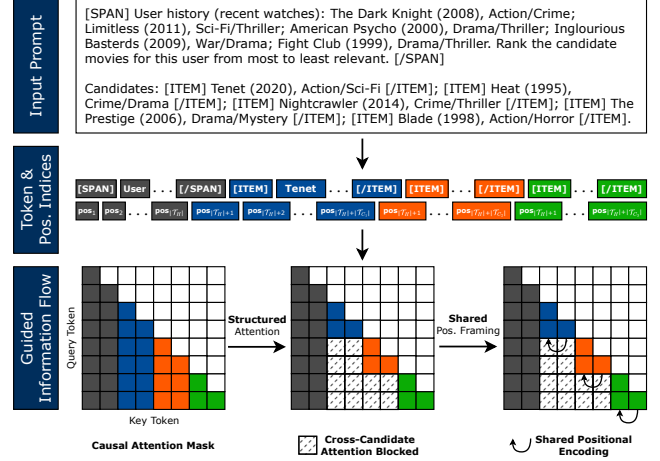
However, recent work shows that listwise LLM rerankers can produce different rankings for the same candidate set when the input order is changed [3, 13, 16]. This behavior arises from architectural properties of autoregressive transformers, including causal attention and positional encodings [3], which couple predicted relevance to candidate position. As a result, ranking outputs can depend on prompt serialization rather than only on the underlying items and user context.

Prior approaches aim to mitigate this effect but do not remove its source. Inference-time methods such as permutation aggregation and greedy selection improve stability by combining multiple candidate orders, but require repeated model calls [1, 13]. Post-hoc calibration methods adjust scores to counteract position bias after scoring has already been affected [18]. Training-time regularization and position-aware fine-tuning penalize disagreement across permutations, but the model still has access to order information during computation [2, 29]. These approaches reduce variability in outputs, but operate outside the model’s core computation and leave the underlying order dependence intact.

A notable training-time method is ALRO [2], which encourages invariance through a loss-based formulation by casting ranking as a generative task (e.g., producing sequences such as "A > B > C"). While effective in explicit-feedback settings, ALRO differs from our setting in two key aspects. It does not operate in a two-stage reranking pipeline over retrieved candidates, and it enforces invariance through training objectives rather than model computation.

In contrast, we enforce permutation invariance directly within the scoring architecture. Our approach removes order-dependent

Figure 1: Overview of InvariRank. Structured attention blocks cross-candidate interactions, and shared positional framing aligns positional offsets across candidates.



computation in a single forward pass, rather than reducing its effects through aggregation, calibration, or regularization.

3 Methodology

We formulate the LLM-based recommendation task as a permutation-equivariant listwise scoring problem, where each candidate receives the same score regardless of input order. Since recommendation operates over sets, scores should not depend on how candidates are serialized. We therefore design the model to produce permutation-invariant scores while retaining listwise supervision via a LambdaRank objective, and introduce InvariRank as a corresponding framework.

3.1 Permutation-Equivariant Listwise Ranking

Let $C = \{c_1, \dots, c_N\}$ denote a candidate set and π a permutation over its indices. A reranker should satisfy permutation equivariance

$$s_{\pi(i)}(\mathcal{H}, \pi(C)) = s_i(\mathcal{H}, C), \quad (1)$$

meaning that reordering the input candidates does not change the score assigned to each item. The induced ranking is therefore consistent under permutation.

For each user query, we construct a single input sequence as

$$q = \text{concat} \left(\underbrace{[\text{SPAN}] \mathcal{I} \parallel \mathcal{H} [\text{/SPAN}]}_{\text{shared context}}, \underbrace{[\text{ITEM}] c_1 [\text{/ITEM}] \dots [\text{ITEM}] c_N [\text{/ITEM}]}_{\text{candidate set}} \right) \quad (2)$$

where $\mathcal{I}$ is the instruction, $\mathcal{H}$ is user history, and C is the candidate set.

We treat the LLM as a scoring function f_θ . For each candidate c_i , we compute a scalar score s_i as the mean token-level log-probability over its [ITEM] span. Candidates are ranked in a single forward pass.

However, standard decoder-only LLMs do not satisfy permutation equivariance.

3.2 Order Dependence in Decoder-Only LLMs

Under standard causal attention and positional encoding, candidate scores take the form

$$s_i = f_\theta(\mathcal{H}, c_i, c_{<i}, p_i), \quad (3)$$

where $c_{<i}$ denotes preceding candidates and p_i the serialized position. This introduces two sources of order dependence including (i) cross-candidate attention leakage, and (ii) position-dependent encoding under RoPE.

As a result, permuting the input sequence can change scores, violating permutation equivariance.

3.3 Structured Attention for Candidate Isolation

To remove cross-candidate interference, we impose a structured segment mask. Let $\mathcal{T}_H$ denote token indices of the shared context and $\mathcal{T}_i$ those of candidate c_i . We define

$$M_{t,u}^{\text{seg}} = \mathbb{I}\left[(t, u) \in (\mathcal{T}_H \times \mathcal{T}_H) \cup \bigcup_{i=1}^N (\mathcal{T}_i \times \mathcal{T}_i) \cup \bigcup_{i=1}^N (\mathcal{T}_i \times \mathcal{T}_H)\right] \quad (4)$$

and apply $M = M^{\text{causal}} \wedge M^{\text{seg}}$.

This restricts each candidate to attend only to the shared context and itself under the causal mask, yielding

$$s_i = g_\theta(\mathcal{H}, c_i), \quad (5)$$

removing dependence on other candidates.

3.4 Shared Positional Framing

Even with isolated attention, positional encodings can introduce order dependence. To address this, we assign positions using a shared frame.

Tokens in $\mathcal{T}_H$ receive positions $1, \dots, |\mathcal{T}_H|$. Each candidate segment $\mathcal{T}_i$ is assigned

$$|\mathcal{T}_H| + 1, \dots, |\mathcal{T}_H| + |\mathcal{T}_i|$$

independent of i and the permutation.

For any permutation π , relative offsets between candidate and context tokens are preserved

$$p_\pi(u) - p_\pi(t) = p(u) - p(t), \quad \forall t \in \mathcal{T}_H, u \in \mathcal{T}_i. \quad (6)$$

Thus, each candidate is evaluated under an identical positional relationship to the context, ensuring invariance.

3.5 Listwise Training Objective

Although candidate representations are conditionally independent during the forward pass, training remains listwise. We optimize scores jointly using a LambdaRank-style pairwise logistic loss. For each pair (i, j) with $y_i > y_j$, the loss is

$$\ell_{ij} = |\text{AnDCG}_{ij}| \log(1 + \exp(-\sigma(s_i - s_j))). \quad (7)$$

The final loss is obtained by averaging over all preference pairs. This weights each pairwise preference by the change in nDCG induced by swapping the two candidates.

This couples candidate scores through the loss, preserving listwise supervision while ensuring that permutation invariance is enforced by the model computation rather than the objective.

4 Experiment Setup

Datasets. We evaluate on two datasets: MovieLens-32M [8] and Amazon Books [11], both with timestamped explicit-feedback interactions. For each user, interactions are sorted chronologically and split into 70%/10%/20% train/validation/test segments. Each query uses the 20 most recent past interactions as history, and relevance is derived from future interactions to avoid temporal leakage.

Candidate Generation. We use LightGCN as a first-stage retriever trained on past interactions with ratings ≥ 4 as implicit positives. For each query, we construct a reranking list of size $K = 25$ from a retrieved candidate pool, ensuring future positives are included when available, and filling remaining slots with retrieved or sampled non-interacted items. All methods rerank the same candidate sets, with non-interacted items assigned relevance 0.

Implementation. We fine-tune LLaMA 3.2 3B-Instruct [24] and Mistral 7B-Instruct [14] using LoRA (rank 16, $\alpha = 32$). Each candidate is scored using the mean token-level log-probability over its span. Models are trained with a listwise LambdaRank objective that optimizes pairwise swaps weighted by ΔnDCG .

We use a maximum sequence length of 4096 and truncate histories to 20 interactions. Training is performed for 500 optimizer steps with batch size 16 (via gradient accumulation), learning rate 5×10^{-5} , and AdamW with 5% warmup. Hyperparameters are selected using validation nDCG@10.

For LFT, we use standard causal attention and positional encoding. InvariRank replaces these with structured segment masking and shared positional framing.

Baselines. We compare against inference-time, post-hoc, and fine-tuned baselines. **Zero-shot** applies an LLM reranker without fine-tuning [4, 15]. **SGS** constructs rankings greedily with multiple forward passes [1]. **Bootstrapping** aggregates rankings over permutations [13]. **STELLA** applies post-hoc calibration using a learned position-bias matrix [18]. We adapt STELLA to produce ranked lists in our setting. It is designed to improve ranking effectiveness under position bias, rather than to enforce consistency in the induced preference structure. **LFT** denotes listwise fine-tuning with the same scoring function and LambdaRank objective as InvariRank but without architectural invariance constraints.

Evaluation. We evaluate ranking effectiveness using HR@ k and nDCG@ k for $k \in \{5, 10\}$. Permutation robustness is measured using Kendall's τ , Spearman's ρ , and top- k agreement across rankings produced from different permutations of the same candidate set. To assess sensitivity, each candidate set is evaluated under multiple permutations and metrics are averaged across permutations.

5 Results

Table 1 reports recommendation performance and permutation robustness on MovieLens-32M and Amazon Books. We compare InvariRank against prompting-based, post-hoc, and fine-tuned baselines.

Main results. Listwise fine-tuning (LFT) achieves the strongest ranking effectiveness across both datasets, substantially improving HR and nDCG over inference-time baselines. However, it remains sensitive to candidate ordering, with lower Kendall's τ , Spearman's ρ , and top- k agreement. In contrast, InvariRank maintains competitive effectiveness while achieving near-perfect permutation

Table 1: Reranking results on MovieLens-32M & Amazon Books with list length $K = 25$. HR@ k and nDCG@ k denote Hit Rate and nDCG at $k \in \{5, 10\}$, and Kendall’s τ , Spearman’s ρ , and top- k agreement (T@5) measure permutation robustness. Higher is better for all metrics.

Model	Method	ML-32M								Books							
		HR@5	HR@10	nDCG@5	nDCG@10	$\tau \uparrow$	$\rho \uparrow$	T@5 $\uparrow$	HR@5	HR@10	nDCG@5	nDCG@10	$\tau \uparrow$	$\rho \uparrow$	T@5 $\uparrow$		
LLaMA-3B	Zero-shot	0.4796	0.7323	0.1594	0.2384	0.5364	0.6665	0.6661	0.1426	0.3483	0.0886	0.1540	0.5984	0.7141	0.6828		
	Bootstrapping	0.4986	0.7393	0.1660	0.2448	0.6892	0.8490	0.6596	0.1683	0.3623	0.1051	0.1669	0.7279	0.8685	0.6748		
	STELLA	0.5143	0.7936	0.1744	0.2624	0.1176	0.1688	0.2166	0.2309	0.4313	0.1410	0.2048	0.0328	0.0476	0.2084		
	SGS	0.4993	0.7570	0.1654	0.2456	0.7705	0.9118	0.7476	0.1270	0.3490	0.0808	0.1515	<u>0.8081</u>	<u>0.9106</u>	<u>0.8084</u>		
	LFT	0.9883	1.0000	0.7679	0.8486	<u>0.8861</u>	<u>0.9708</u>	<u>0.8389</u>	0.4806	0.7096	0.3364	0.4108	0.7758	0.9020	0.7954		
	InvariRank	<u>0.9700</u>	1.0000	<u>0.7260</u>	<u>0.8166</u>	0.9883	0.9984	0.9906	<u>0.4720</u>	<u>0.6833</u>	<u>0.3184</u>	<u>0.3871</u>	0.9834	0.9977	0.9805		
Mistral-7B	Zero-shot	0.4480	0.7256	0.1561	0.2332	0.4508	0.5927	0.5406	0.1750	0.3513	0.1127	0.1688	0.5523	0.6944	0.6167		
	Bootstrapping	0.4530	0.7246	0.1572	0.2352	0.6274	0.7964	0.5869	0.1936	0.3616	0.1236	0.1771	0.7018	0.8545	0.6600		
	STELLA	0.5180	0.8119	0.1719	0.2624	0.0111	0.0160	0.2053	0.2243	0.4243	0.1428	0.2070	0.0332	0.0475	0.2510		
	SGS	0.5110	0.7526	0.1786	0.2492	0.6596	0.8287	0.5758	0.1863	0.3666	0.1184	0.1758	<u>0.7432</u>	<u>0.8786</u>	0.7198		
	LFT	0.9936	1.0000	0.7985	0.8698	<u>0.8769</u>	<u>0.9667</u>	<u>0.8419</u>	0.5626	0.7396	0.4049	0.4623	0.7407	0.8750	<u>0.7602</u>		
	InvariRank	<u>0.9766</u>	1.0000	<u>0.7722</u>	<u>0.8523</u>	0.9936	0.9992	0.9923	<u>0.5216</u>	<u>0.6993</u>	<u>0.3591</u>	<u>0.4166</u>	0.9846	0.9979	0.9843		

robustness without permutation-based training or ensembling. For example, on MovieLens-32M with LLaMA-3B, InvariRank attains $\tau = 0.9883$ and $\rho = 0.9984$ compared to 0.8861 and 0.9708 for LFT, while retaining comparable nDCG@10 (0.8166 vs. 0.8486). Similar trends hold across both backbones and datasets.

Comparison to robustness baselines. Inference-time methods such as bootstrapping and SGS improve robustness over zero-shot prompting, but require multiple forward passes and still fall short of InvariRank. STELLA applies post-hoc calibration and is adapted here to produce ranked lists in our setting. Its lower rank correlation reflects its design to improve ranking effectiveness under position bias, rather than to enforce consistency in the induced preference ordering. In contrast, InvariRank enforces invariance within the model computation and achieves higher robustness across all metrics in a single forward pass.

Effectiveness–robustness trade-off. We observe a consistent trade-off between ranking effectiveness and permutation robustness. While LFT achieves the highest nDCG, its rankings vary under permutation. InvariRank incurs a small reduction in effectiveness but produces highly stable rankings, with large gains in τ , ρ , and top- k agreement. This shows that architectural invariance removes position-induced variability that standard fine-tuning does not address.

Position bias and exposure. Figure 2 shows top- k exposure by input position. Zero-shot and bootstrapping exhibit strong position bias, while LFT reduces but does not eliminate it. SGS and STELLA

substantially flatten exposure, showing that inference-time methods can mitigate position bias. InvariRank achieves similarly uniform exposure while also providing near-perfect permutation robustness, indicating that architectural invariance better removes residual order dependence.

Table 2: Architectural ablation on ML-32M using LLaMA 3.2 3B Instruct, isolating the contribution of each component.

Method	HR@5	nDCG@5	τ	ρ	T@5
InvariRank _{Pos}	0.6423	0.2465	0.5389	0.6967	0.5429
InvariRank _{Attn}	0.9690	0.7298	0.9193	0.9822	0.9097
InvariRank _{Full}	0.9700	0.7260	0.9883	0.9984	0.9906

6 Ablation

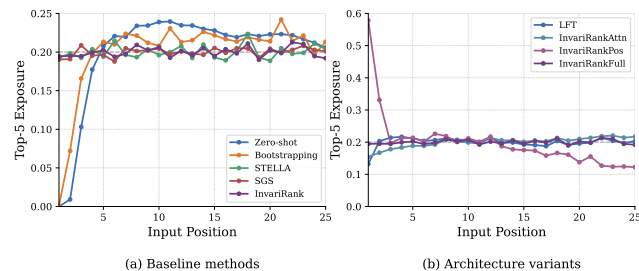
Table 2 evaluates the contribution of structured attention masking and shared positional framing on MovieLens-32M with LLaMA 3.2 3B. Figure 2(b) shows the corresponding exposure patterns.

Using only shared positional framing (InvariRank_{Pos}) leads to poor effectiveness and weak robustness ($\tau = 0.5389$, $\rho = 0.6967$), with strong position bias in exposure. In contrast, structured attention masking (InvariRank_{Attn}) recovers strong ranking performance and substantially improves robustness ($\tau = 0.9193$, $\rho = 0.9822$), indicating that preventing cross-candidate interference is the main driver of invariance.

Combining both components (InvariRank_{Full}) achieves the best results, with near-perfect robustness ($\tau = 0.9883$, $\rho = 0.9984$) and uniform exposure. This shows that attention masking removes most order dependence, while shared positional framing further refines invariance.

7 Discussion

Our results highlight a fundamental trade-off between ranking effectiveness and permutation robustness in LLM-based reranking. While listwise fine-tuning (LFT) improves effectiveness, it does not eliminate order dependence, leading to inconsistent rankings under permutation. In contrast, InvariRank enforces permutation invariance at the architectural level, producing stable rankings with near-perfect agreement while maintaining competitive effectiveness. This suggests that architectural constraints, rather than training objectives alone, are necessary for reliable listwise ranking.

Figure 2: Top-5 exposure by input position on ML-32M using LLaMA 3.2 3B-Instruct with list length $K = 25$.

A key limitation of InvariRank is that candidate isolation removes cross-candidate interactions, potentially discarding useful comparative signals and limiting fine-grained preference modeling. Additionally, our evaluation focuses on fixed-size candidate sets and two recommendation datasets, leaving open how the approach scales to larger candidate sets or generalizes to other ranking tasks such as information retrieval and retrieval-augmented generation (RAG). Future work should explore hybrid designs that incorporate controlled cross-candidate interaction while preserving permutation invariance, as well as broader evaluation across domains and candidate set sizes.

8 Conclusion

We study position bias in listwise LLM-based recommendation reranking, where candidate permutations can change item scores and rankings. We propose InvariRank, an order-invariant listwise reranker that enforces permutation robustness architecturally through structured segment masking and shared positional framing, without permutation ensembling or auxiliary invariance losses. Evaluations on MovieLens-32M and Amazon Books with Mistral 7B and LLaMA 3.2 3B show that InvariRank maintains competitive ranking effectiveness while achieving near-perfect permutation robustness, measured by Kendall's τ , Spearman's ρ , and top- k agreement. These results show that eliminating order dependence by design is a practical route to stable, single-pass LLM reranking for recommendation.

References

- [1] Ethan Bito, Yongli Ren, and Estrid He. 2025. Evaluating Position Bias in Large Language Model Recommendations. arXiv:2508.02020 [cs.IR] <https://arxiv.org/abs/2508.02020>
- [2] Wen-Shuo Chao, Zhi Zheng, Hengshu Zhu, and Hao Liu. 2024. Make Large Language Model a Better Ranker. arXiv:2403.19181 [cs.IR] <https://arxiv.org/abs/2403.19181>
- [3] Florin Cuconasu, Simone Filice, Guy Horowitz, Yoelle Maarek, and Fabrizio Silvestri. 2025. Do RAG Systems Really Suffer From Positional Bias? arXiv:2505.15561 [cs.CL] <https://arxiv.org/abs/2505.15561>
- [4] Sunhao Dai, Ninglu Shao, Haiyuan Zhao, Weijie Yu, Zihua Si, Chen Xu, Zhongxiang Sun, Xiao Zhang, and Jun Xu. 2023. Uncovering ChatGPT's Capabilities in Recommender Systems. In *Proceedings of the 17th ACM Conference on Recommender Systems (RecSys '23)*. ACM, 1126–1132. doi:10.1145/3604915.3610646
- [5] Li Dong, Nan Yang, Wenhui Wang, Furu Wei, Xiaodong Liu, Yu Wang, Jianfeng Gao, Ming Zhou, and Hsiao-Wuen Hon. 2019. Unified language model pre-training for natural language understanding and generation. *Advances in neural information processing systems* 32 (2019).
- [6] Jingtong Gao, Bo Chen, Weiwen Liu, Xiangyang Li, Yichao Wang, Wanyu Wang, Huifeng Guo, Ruiming Tang, and Xiangyu Zhao. 2025. LLM4Rerank: LLM-based Auto-Reranking Framework for Recommendations. arXiv:2406.12433 [cs.IR] <https://arxiv.org/abs/2406.12433>
- [7] Shijie Geng, Shuchang Liu, Zuohui Fu, Yingqiang Ge, and Yongfeng Zhang. 2022. Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5). In *Proceedings of the 16th ACM conference on recommender systems*. 299–315.
- [8] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article 19 (Dec. 2015), 19 pages. doi:10.1145/2827872
- [9] Jiayuan He, Jianzhong Qi, and Kotagiri Ramamohanarao. 2019. A joint context-aware embedding for trip recommendations. In *2019 IEEE 35th international conference on data engineering (ICDE)*. IEEE, 292–303.
- [10] Jiayuan He, Jianzhong Qi, and Kotagiri Ramamohanarao. 2020. Timesan: A time-modulated self-attentive network for next point-of-interest recommendation. In *2020 International joint conference on neural networks (IJCNN)*. IEEE, 1–8.
- [11] Yupeng Hou, Jiacheng Li, Zhankui He, An Yan, Xiushi Chen, and Julian McAuley. 2024. Bridging Language and Items for Retrieval and Recommendation. arXiv preprint arXiv:2403.03952 (2024).
- [12] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large language models are zero-shot rankers for recommender systems. In *European Conference on Information Retrieval*. Springer, 364–381.
- [13] Yupeng Hou, Junjie Zhang, Zihan Lin, Hongyu Lu, Ruobing Xie, Julian McAuley, and Wayne Xin Zhao. 2024. Large Language Models are Zero-Shot Rankers for Recommender Systems. arXiv:2305.08845 [cs.IR] <https://arxiv.org/abs/2305.08845>
- [14] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, Léo Renard Lavaud, Marie-Anne Lachaux, Pierre Stock, Teven Le Scao, Thibaut Lavril, Thomas Wang, Timothée Lacroix, and William El Sayed. 2023. Mistral 7B. arXiv:2310.06825 [cs.CL] <https://arxiv.org/abs/2310.06825>
- [15] Junling Liu, Chao Liu, Peilin Zhou, Renjie Lv, Kang Zhou, and Yan Zhang. 2023. Is ChatGPT a Good Recommender? A Preliminary Study. arXiv:2304.10149 [cs.IR] <https://arxiv.org/abs/2304.10149>
- [16] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 8086–8098.
- [17] Sichun Luo, Bowei He, Haohan Zhao, Wei Shao, Yanlin Qi, Yinya Huang, Aojun Zhou, Yuxuan Yao, Zongpeng Li, Yuanzhang Xiao, Mingjie Zhan, and Linqi Song. 2024. RecRanker: Instruction Tuning Large Language Model as Ranker for Top-k Recommendation. arXiv:2312.16018 [cs.IR] <https://arxiv.org/abs/2312.16018>
- [18] Tianhui Ma, Yuan Cheng, Hengshu Zhu, and Hui Xiong. 2023. Large Language Models are Not Stable Recommender Systems. arXiv:2312.15746 [cs.IR] <https://arxiv.org/abs/2312.15746>
- [19] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-Shot Listwise Document Reranking with a Large Language Model. arXiv:2305.02156 [cs.IR] <https://arxiv.org/abs/2305.02156>
- [20] Xueguang Ma, Xinyu Zhang, Ronak Pradeep, and Jimmy Lin. 2023. Zero-shot listwise document reranking with a large language model. arXiv preprint arXiv:2305.02156 (2023).
- [21] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing* 568 (2024), 127063.
- [22] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 14918–14937.
- [23] Raphael Tang, Crystina Zhang, Xueguang Ma, Jimmy Lin, and Ferhan Türe. 2024. Found in the middle: Permutation self-consistency improves listwise ranking in large language models. In *Proceedings of the 2024 conference of the North American chapter of the Association for Computational Linguistics: human language technologies (volume 1: long papers)*. 2327–2340.
- [24] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. LLaMA: Open and Efficient Foundation Language Models. arXiv:2302.13971 [cs.CL] <https://arxiv.org/abs/2302.13971>
- [25] Wei Wei, Xubin Ren, Jiabin Tang, Qinyong Wang, Lixin Su, Suqi Cheng, Junfeng Wang, Dawei Yin, and Chao Huang. 2024. Llmrec: Large language models with graph augmentation for recommendation. In *Proceedings of the 17th ACM international conference on web search and data mining*. 806–815.
- [26] Haobo Zhang, Qianan Zhu, and Zhicheng Dou. 2025. Enhancing Reranking for Recommendation with LLMs through User Preference Retrieval. In *Proceedings of the 31st International Conference on Computational Linguistics*, Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert (Eds.). Association for Computational Linguistics, Abu Dhabi, UAE, 658–671. <https://aclanthology.org/2025.coling-main.45/>
- [27] Shuai Zhang, Lina Yao, Aixin Sun, and Yi Tay. 2019. Deep learning based recommender system: A survey and new perspectives. *ACM computing surveys (CSUR)* 52, 1 (2019), 1–38.
- [28] Xinyu Zhang, Sebastian Hofstätter, Patrick Lewis, Raphael Tang, and Jimmy Lin. 2023. Rank-without-GPT: Building GPT-Independent Listwise Rerankers on Open-Source Large Language Models. arXiv:2312.02969 [cs.CL] <https://arxiv.org/abs/2312.02969>
- [29] Zheng Zhang, Fan Yang, Ziyang Jiang, Zheng Chen, Zhengyang Zhao, Chengyuan Ma, Liang Zhao, and Yang Liu. 2024. Position-Aware Parameter Efficient Fine-Tuning Approach for Reducing Positional Bias in LLMs. arXiv:2404.01430 [cs.CL] <https://arxiv.org/abs/2404.01430>
- [30] Tianqing Zhu, Yongli Ren, Wanlei Zhou, Jia Rong, and Ping Xiong. 2014. An effective privacy preserving algorithm for neighborhood-based collaborative filtering. *Future Generation Computer Systems* 36 (2014), 142–155.
- [31] Shengyao Zhuang, Honglei Zhuang, Bevan Koopman, and Guido Zuccon. 2024. A Setwise Approach for Effective and Highly Efficient Zero-shot Ranking with Large Language Models. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR 2024)*. ACM, 38–47. doi:10.1145/3626772.3657813